

BÁO CÁO SO SÁNH CÁC KỸ THUẬT NOTIFICATION TRÊN WEB

Short Polling · Long Polling · Server-Sent Events (SSE) · WebSocket · Web Push

1. Executive Summary

Không có một transport duy nhất phù hợp cho mọi notification. Với thông báo không khẩn cấp và traffic nhỏ, Short Polling có chi phí hạ tầng và độ phức tạp thấp nhất nhưng đổi lại bằng request rỗng và latency phụ thuộc interval. Long Polling giảm request rỗng và đạt latency gần real-time hơn, nhưng có thể giữ connection/thread lâu nếu server dùng blocking I/O. SSE phù hợp cho luồng server→client khi browser đang mở, vì dùng một HTTP connection dài và EventSource có auto-reconnect. WebSocket phù hợp khi cần giao tiếp hai chiều tần suất cao, nhưng chi phí vận hành tăng rõ khi self-host ở production scale lớn vì phải quản lý connection lifecycle, heartbeat, reconnect và broadcast liên instance. Web Push là một bài toán khác: nó dành cho background/OS-level notification và đi qua Push Service bên thứ ba, nên server của ứng dụng không phải duy trì connection liên tục.

Theo tài liệu nghiên cứu, production thường kết hợp nhiều kỹ thuật: SSE cho in-app real-time, Web Push khi user offline/tab đóng, và REST/DB cho lịch sử/recovery. Lựa chọn cuối cùng cần được xác nhận bằng implementation + benchmark trên workload thực tế.

2. Ma trận so sánh tổng quan

Tiêu chí	Short Polling	Long Polling	SSE	WebSocket	Web Push
Mô hình	Client pull định kỳ	Client pull, request được giữ mở	Server → client	Hai chiều	Server → Push Service → client
Realtime	Thấp	Trung bình-Cao	Cao	Cao nhất trong nhóm theo tài liệu	Không đồng bộ, best-effort
Connection	Không giữ lâu	Giữ request mở	1 connection dài	1 connection dài, 2 chiều	Không giữ connection liên tục
Server resource	Thấp/request nhưng tăng theo user × poll	Cao nếu blocking I/O	Trung bình	Trung bình-Cao	Thấp ở connection layer
Network efficiency	Kém: nhiều response rỗng	Khá	Tốt	Tốt	Rất tốt: chỉ gửi khi có event
Multi-instance	Dễ/stateless	Trung bình	Cần sticky/pub-sub theo thiết kế	Cần Redis/broker khi nhiều instance	Dễ ở connection layer
Client reconnect	Tự implement	Tự implement	Built-in	Tự implement	Không áp dụng cho persistent connection
Background/browser đóng	Không	Không	Không	Không	Có
Infrastructure complexity	Thấp	Thấp-Trung bình	Trung bình	Cao	Trung bình
Implementation complexity	Thấp	Trung bình	Thấp-Trung bình	Cao	Cao

Bảng trên là qualitative assessment tổng hợp từ tài liệu nghiên cứu, không phải benchmark đo trực tiếp của project.

3. Chi phí: cần hiểu là Total Cost of Ownership, không chỉ tiền hosting

Kỹ thuật	Chi phí hạ tầng trực tiếp	Driver làm chi phí tăng	Đánh giá định tính
Short Polling	Thấp khi traffic nhỏ	Request rate = users × polling frequency; bandwidth/CPU/DB query tăng theo poll	Thấp nhất cho workload nhỏ, không hiệu quả khi cần polling dày
Long Polling	Thấp-Trung bình	Concurrent waiting requests; blocking I/O có thể làm tăng thread/memory pressure	Rẻ nếu async/non-blocking; đắt hơn khi triển khai blocking

Kỹ thuật	Chi phí hạ tầng trực tiếp	Driver làm chi phí tăng	Đánh giá định tính
SSE	Trung bình	Số connection dài, memory/socket overhead; proxy/load balancer; multi-instance broadcast	Chi phí hợp lý cho 1 chiều nếu platform hỗ trợ persistent HTTP
WebSocket	Trung bình-Cao	Concurrent connections, heartbeat, reconnect storm, multi-instance broker, WS-aware LB/proxy, observability	TCO cao hơn nếu self-host ở scale lớn
Web Push	Thấp ở connection layer; có dependency Push Service	Số subscription, send rate, application-side subscription management	Không cần giữ connection; chi phí vận hành server thấp cho notification không thường xuyên

Lưu ý về pricing: tài liệu nguồn không cung cấp bảng giá hiện hành cụ thể cho từng cloud/provider. Vì vậy báo cáo không bịa đơn giá. Các nhận định “thấp/cao” ở đây là relative TCO dựa trên connection, request, network và operational complexity.

4. Sẵn có hay cần service riêng?

Kỹ thuật	Browser/client API	Có cần service riêng?	Ghi chú production
Short Polling	HTTP/REST, browser fetch	Không	Có thể chạy trên hầu hết hosting/proxy/LB
Long Polling	HTTP/REST	Không	Không cần protocol service riêng; server nên hỗ trợ async/non-blocking nếu scale
SSE	EventSource	Không	Cần HTTP streaming/persistent connection; proxy buffering và timeout phải được kiểm soát
WebSocket	WebSocket API	Không bắt buộc	Có thể self-host; multi-instance thường cần LB/proxy phù hợp và Redis/pub-sub hoặc broker
Web Push	Push API + Service Worker + Notification API	Có Push Service trung gian	Browser cấp PushSubscription; application server gửi qua Push Service bằng VAPID

5. Khả năng chịu tải server và cơ chế scale

5.1 Short Polling — tải theo request rate

Server không phải giữ connection lâu. Vì vậy mỗi request tương đối độc lập và dễ scale horizontally/stateless. Tuy nhiên tải tăng gần tuyến tính theo số user $\times$ tần suất poll; phần lớn request có thể trả về empty response. Khi muốn latency thấp hơn bằng cách giảm interval, request rate tăng và chi phí CPU/network/DB cũng tăng. Đây là trade-off cốt lõi của Short Polling.

5.2 Long Polling — giảm request rỗng nhưng chuyển tải sang concurrent waiting connections

Long Polling giảm đáng kể số request rỗng và latency gần real-time. Đối lại server phải giữ request đang chờ. Nếu runtime dùng blocking/thread-per-request, nhiều connection đồng thời có thể làm cạn thread pool. Non-blocking I/O giúp mô hình này chịu tải tốt hơn, nhưng lifecycle phức tạp hơn: timeout, cancellation, retry, duplicate request và dedupe event phải được xử lý.

5.3 SSE — một connection dài cho mỗi client

SSE giữ một HTTP connection dài và stream event server→client. Vì vậy không còn polling overhead, nhưng server phải quản lý nhiều long-lived connections. Tài liệu xếp server resource usage ở mức trung bình. Khi có nhiều instance, cần cơ chế broadcast liên instance; tài liệu nêu sticky/pub-sub là các hướng cần cân nhắc. HTTP/1.1 còn có giới hạn connection/domain ở browser; HTTP/2 multiplexing giải quyết vấn đề này tốt hơn.

5.4 WebSocket — scale connection state và broadcast

WebSocket giữ một connection TCP full-duplex cho mỗi client. Điểm khó không chỉ là số connection mà là state của connection: authentication, heartbeat/ping-pong, stale connection detection, reconnect, backpressure, graceful shutdown và routing message. Khi có nhiều server instance, client A có thể nằm ở server A nhưng event được tạo ở server B; do đó cần broadcast liên instance, thường qua Redis Pub/Sub hoặc message broker. Load balancer/proxy cũng phải hỗ trợ HTTP Upgrade và có thể cần sticky session hoặc pub/sub layer.

5.5 Web Push — đẩy tải delivery connection sang Push Service

Web Push không yêu cầu application server giữ persistent connection tới browser. Application server gửi payload tới Push Service của browser; Push Service đánh thức Service Worker, kể cả khi tab đã đóng trong các điều kiện được hỗ trợ. Vì vậy ở connection layer, server của ứng dụng có gánh nặng thấp hơn SSE/WebSocket. Đối lại, delivery latency/reliability phụ thuộc một phần vào Push Service và semantics là best-effort theo tài liệu.

6. Self-host WebSocket và production scale lớn

Self-host WebSocket là khả thi và không bắt buộc phải mua managed realtime service. Có thể triển khai WebSocket server trực tiếp trên runtime tương ứng, phía trước là load balancer/reverse proxy. Tuy nhiên “self-host được” không có nghĩa là “chỉ cần mở WebSocket server là scale xong”.

Kiến trúc tối thiểu: Browser → Load Balancer/Proxy → WebSocket Gateway instances → Application/Event layer → Redis Pub/Sub hoặc Message Broker → các Gateway instances.

Khi scale ngang: mỗi instance chỉ trực tiếp biết các connection mà nó đang giữ. Nếu event phát sinh ở instance B nhưng recipient đang nối vào instance A, cần cross-instance fan-out. Đây là lý do tài liệu đánh giá WebSocket multi-instance phức tạp và thường cần Redis/broker.

Điểm nghẽn cần benchmark: concurrent connections, connection churn, reconnect storm, message fan-out, heartbeat traffic, payload size, slow clients/backpressure, CPU, memory, bandwidth và broker throughput. Không thể kết luận “X connections/server” nếu chưa xác định runtime, CPU/RAM, payload, message rate, network và workload.

Managed realtime service: tài liệu liệt kê Ably, Pusher và PubNub như các dịch vụ trừu tượng hóa WebSocket/SSE và cung cấp các capability như delivery, presence, history, horizontal scaling. Trade-off là chuyển một phần operational complexity sang chi phí dịch vụ và dependency bên ngoài.

7. Phân tích riêng từng kỹ thuật

Short Polling

Use case: Thông báo không khẩn cấp, traffic nhỏ.

Đặc tính: HTTP thuần, stateless, dễ debug, tương thích hosting/proxy tốt; nhược điểm là request rỗng và latency phụ thuộc interval.

Server/load: Server không giữ connection lâu nhưng request volume tăng theo $\text{users} \times \text{polling frequency}$.

Long Polling

Use case: Latency thấp trong môi trường proxy/firewall khó tính.

Đặc tính: Ít request rỗng hơn, gần real-time, HTTP thuần; lifecycle phức tạp và có nguy cơ thread exhaustion nếu blocking I/O.

Server/load: Phụ thuộc mạnh vào async/non-blocking model.

SSE

Use case: Server→client một chiều, dashboard/feed/activity/notification khi browser mở.

Đặc tính: EventSource built-in, auto-reconnect, Last-Event-ID, HTTP/HTTPS; một chiều và cần xử lý proxy buffering/timeout.

Server/load: Tải chính là long-lived connections; multi-instance cần broadcast phù hợp.

WebSocket

Use case: Chat, collaborative editing, multiplayer, presence/typing, agent control.

Đặc tính: Full-duplex trên một connection; đổi lại operational complexity cao: heartbeat/reconnect/backpressure/multi-instance/LB.

Server/load: Self-host được nhưng scale lớn cần gateway + broker/pub-sub và benchmark connection/fan-out.

Web Push

Use case: Notification khi user offline/tab đóng, OS-level notification.

Đặc tính: Background/browser-closed support; cần HTTPS, permission, Service Worker, VAPID và subscription lifecycle; delivery best-effort.

Server/load: Giảm connection burden cho application server nhưng phụ thuộc Push Service.

8. Reliability, security và operational complexity

Tiêu chí	Short Polling	Long Polling	SSE	WebSocket	Web Push
Reconnect	Tự implement	Tự implement	Built-in	Tự implement	Subscription lifecycle
Delivery recovery	Có thể query DB	Có thể query DB	Event ID + replay	Ack/replay cần tự xây	Best-effort
Backpressure	Request-based	Waiting requests	Stream/client dependent	Quan trọng	Payload/send semantics
Security	REST auth	REST auth	Auth long-lived connection	Handshake + message authorization	VAPID + encrypted payload
Operational risk	Request/DB overload	Timeout/thread exhaustion	Stale connections/proxy buffering	Connection leak/reconnect storm/broker overload	Expired subscriptions/provider behavior

Notification domain nên tách Event / Notification / Delivery Attempt / Client Connection. Business logic không nên phụ thuộc trực tiếp vào transport. Với hệ thống production cần authentication/authorization, rate limiting, structured logging, graceful shutdown, retry/backoff/jitter, duplicate handling và quan sát connection state.

9. Quyết định theo workload

Requirement	Đề xuất	Fit	Lý do
Thông báo không khẩn cấp, traffic nhỏ	Short Polling	Strong fit	Đơn giản, chi phí hạ tầng thấp; chấp nhận latency theo interval
Latency thấp nhưng proxy/firewall khó tính	Long Polling	Strong fit	HTTP thuần, ít request rỗng hơn polling
Server→client, browser đang mở, gần real-time	SSE	Strong fit	Một chiều đúng mô hình; EventSource auto-reconnect
Hai chiều, tần suất cao	WebSocket	Strong fit	Full-duplex trên một connection
Browser/tab đóng, OS notification	Web Push	Strong fit	Background/offline delivery qua Push Service
Nhiều instance + WebSocket	WebSocket + broker/pub-sub	Requires additional infrastructure	Cần fan-out liên instance
Team nhỏ, không muốn vận hành realtime infra	Managed realtime service	Possible	Đổi operational complexity lấy dependency/chi phí dịch vụ

10. Benchmark plan

Benchmark status: Chưa chạy. Báo cáo không chứa số liệu p50/p95/p99, throughput, CPU, memory hoặc max-connections được đo từ project.

Workload: số users/connections; notification rate; payload size; fan-out/fan-in; burst; duration; reconnect storm; slow clients; poor network nếu khả thi.

Metrics: latency p50/p90/p95/p99; notifications/sec; throughput; error rate; reconnect rate; concurrent connections; CPU; memory; bandwidth; queue depth/broker metrics nếu có.

Scenario: A Normal, B Burst, C Fan-out, D High-frequency, E Connection Storm, F Reconnection Storm, G Slow Client, H Poor Network, I Large Payload, J Mixed Workload.

Nguyên tắc: giữ workload/environment giống nhau giữa transports khi có thể; chạy nhiều lần; lưu raw + processed data, configuration/version/command; không overwrite benchmark cũ.

11. Kết luận

Nếu mục tiêu chỉ là notification server→browser khi người dùng đang mở ứng dụng, SSE là điểm xuất phát có TCO và operational complexity cân bằng: không cần service realtime riêng, browser có EventSource, reconnect built-in và phù hợp HTTP infrastructure. Nếu cần bidirectional communication tần suất cao, WebSocket phù hợp hơn nhưng self-host production scale lớn phải được xem như một distributed-systems problem: connection lifecycle + gateway + load balancing + broker/pub/sub + reconnect/backpressure + observability.

Nếu notification phải đến khi browser/tab không mở, Web Push giải quyết một bài toán khác và thường nên được dùng song song với SSE/WebSocket thay vì thay thế chúng. Short Polling và Long Polling vẫn có vị trí khi simplicity hoặc compatibility với hạ tầng cũ quan trọng hơn realtime efficiency.

Quyết định production cần được xác nhận bằng benchmark với workload thực tế và môi trường triển khai cụ thể.

12. Traceability & Sources

Primary source: Notification-Tech-Comparison-Report.pdf — tài liệu nghiên cứu được cung cấp. Đây là nguồn chính cho ma trận qualitative, use cases, scalability, WebSocket self-host/multi-instance và Web Push.

Project methodology: Notification Realtime Research & Engineering Lab — Project Instructions.md — quy định evidence, benchmark, architecture, reliability, security, deployment và reporting.

Referenced in source report: Slack Engineering — “Real-time Messaging”; ByteByteGo; GitHub/go-gitea Issue #36937; WebSocket.org/Ably; RxDB; APIScout; TechPlained; CodeScoop/DEV Community; npmtrends; PkgPulse; MDN Web Docs; các nguồn thống kê Web Push được liệt kê trong tài liệu nghiên cứu.

Evidence classification

FACT / source-derived: protocol characteristics, qualitative comparison, browser/API architecture và production examples được tài liệu nguồn dẫn lại.

INDUSTRY EXPERIENCE: các kiến trúc production và xu hướng adoption được tài liệu nguồn tổng hợp từ engineering/community sources.

EXPERIMENTAL RESULT: chưa có.

Uncertainty: pricing cụ thể, capacity/server cụ thể và benchmark numbers cho project chưa được xác minh/đo; không được suy ra từ report này.